



Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 406 · Computer Security Sessional

TCP Reset Attack on a Video Stream

Final Report

Submitted by

Tamzeed Mahfuz
Gourove Roy

Student ID: 2105012
Student ID: 2105017

Supervised by

Md Zim Mim Siddiquee Sowdha
Lecturer, Department of CSE, BUET

September 18, 2026

Contents

1	Introduction	1
2	Experimental Setup	1
2.1	LAN topology	1
2.2	WAN topology	2
3	Attack Process	2
4	Expected Outcome	2
5	Actual Outcome and Observations	2
5.1	LAN scenario	3
5.2	WAN scenario	6
6	Expected vs. Actual: Differences and Why	9
7	Defence Mechanism	9
7.1	Detection	9
7.2	Prevention	12
7.3	Does the defence work as expected?	12
8	Assumptions	12
9	Conclusion	13

List of Figures

1	LAN topology.	1
2	WAN topology.	2
3	LAN: the video streaming normally from the server (10.36.173.234:8000) before the attack.	3
4	LAN attacker: ARP spoofing the server and the client.	3
5	LAN attacker: injecting the forged TCP resets into the video connection.	4
6	LAN: Wireshark capture showing the ARP poisoning and the injected RST.	4
7	LAN result: the video frozen (buffering spinner) after the reset injection.	5
8	WAN: the video streaming from the remote ngrok server (spinning-nimble-lurch.ngrok-free.dev) over HTTPS.	6
9	WAN: using Wireshark to read the current ngrok edge (server) IP for the video flow.	6
10	WAN attacker: ARP spoofing the client and the gateway (the phone hotspot).	7
11	WAN attacker: injecting the forged resets into the client's HTTPS video connection.	7
12	WAN: Wireshark capture of the injected RST on the port-443 (HTTPS) flow.	8
13	LAN detection: the IDS flags the ARP spoofing (one MAC claiming both victim IPs; the server's MAC changing).	10
14	LAN detection: the injected RST flagged by per-direction TTL (TTL 64 != data TTL 127, minted locally).	10
15	WAN detection: the client-side IDS flags the ARP spoofing of the client and gateway.	11

- 16 LAN prevention: with the peer's MAC pinned PERMANENT ("ARP spoof now ineffective"), the video keeps playing even while the attacker is poisoning. . . . 12

1 Introduction

A **TCP reset (RST) attack** injects a forged TCP segment with the RST flag into an established connection; a host that accepts an acceptable reset aborts the connection immediately. Our target is the TCP connection carrying an HTTP video stream: one accepted reset tears it down and the **video freezes**. To supply a sequence number the receiver will accept, the attacker first becomes a man-in-the-middle (MITM) by **ARP spoofing**.

We implemented the attack and its defence ourselves (Python + Scapy) and demonstrated everything on **real hardware**, in **two topologies**:

- **LAN:** server, client and attacker on one local network (a phone hotspot). The attacker poisons the *server* and the *client*.
- **WAN (realistic):** the server is *remote* (a Flask server on a Kaggle notebook, exposed to the Internet by ngrok over HTTPS); the client and attacker are on a local hotspot. Since the server is on another network, the attacker poisons the *client* and the *router/gateway*. This mirrors a real streaming service reached across the Internet.

Summary of results

In both topologies the attack froze the video on command, including over HTTPS in the WAN case. The defence works too: an IDS detects the attack in both topologies, static ARP fully prevents it on the LAN, and a client-side VPN prevents it on the WAN.

2 Experimental Setup

2.1 LAN topology

Three machines on one phone-hotspot LAN (Figure 1): a **Windows server** running our rate-limited streaming server on port 8000, a **Linux client** playing the video in a browser, and a **Linux attacker** running Scapy. The client reaches the server directly (same subnet), so the attacker poisons both endpoints.

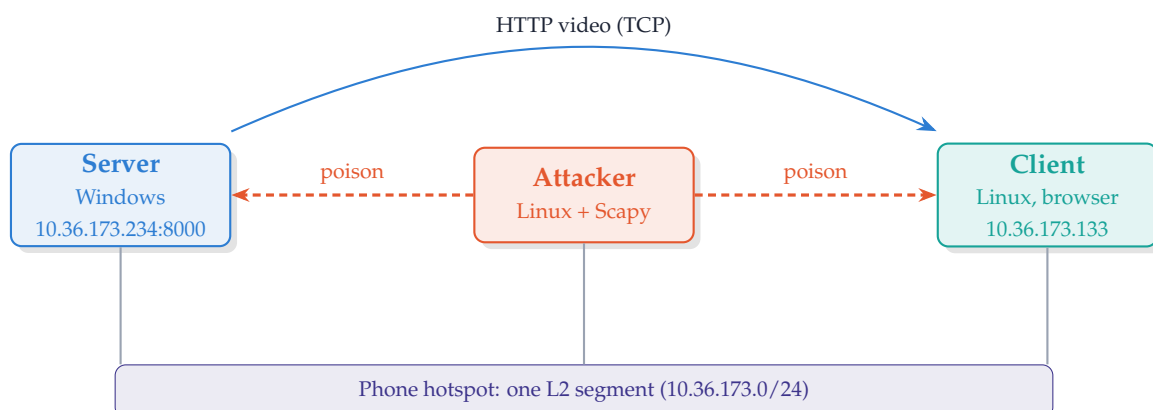


Figure 1: LAN topology: the attacker poisons the server and the client on one hotspot.

2.2 WAN topology

Here the server is remote (Figure 2): a Flask video server in a **Kaggle notebook**, published by **ngrok** at a public HTTPS URL (spinning-nimble-lurch.ngrok-free.dev). The **Linux client** and **Linux attacker** share a phone hotspot whose gateway is the phone itself. The client's video traffic leaves via the gateway, so the attacker poisons the *client* and the *gateway*.

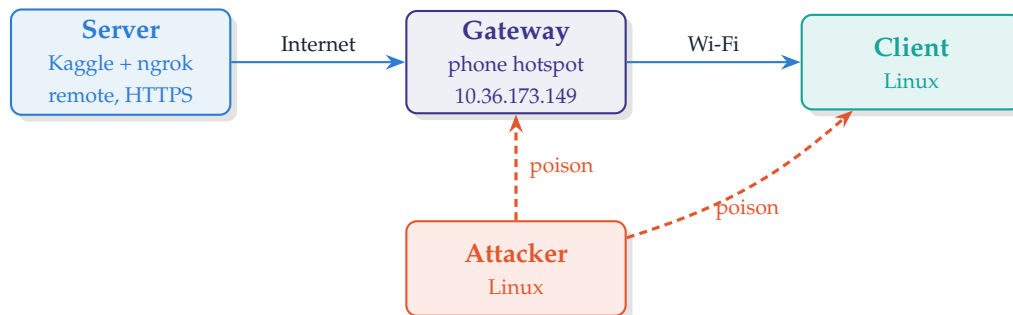


Figure 2: WAN topology: the server is remote; the attacker poisons the client and the gateway (the phone hotspot).

3 Attack Process

The attack runs in four phases:

1. **Poison.** Forged ARP replies make each victim map the other party's IP to the attacker's MAC, so their traffic transits the attacker (the kernel forwards it, so the video keeps playing).
2. **Observe.** The attacker sniffs the video connection. Each packet's TCP header gives exact sequence numbers: a data segment's $\text{seq} + \text{len}$ is the receiver's next expected byte, and a packet's ack field is the sender's own next expected byte.
3. **Forge.** Using those exact values (no guessing), the attacker builds a RST that spoofs the peer, resetting the *sender* with $\text{seq} = \text{ack}$ and the *receiver* with $\text{seq} = \text{seq} + \text{len}$.
4. **Inject.** The reset is sent to the victim; the connection aborts and the video freezes. This works over **HTTPS** because the RST lives in the TCP header, below TLS.

Finding the server IP. On the LAN the server IP is known. On the WAN, ngrok load-balances across several edge IPs and changes them on every restart, so we read the *current* edge IP from a live capture with Wireshark (Figure 9) and target that.

4 Expected Outcome

While the attacker forwards traffic, the video should play normally (the MITM is invisible). On injection, the connection should abort within a fraction of a second, the picture should **freeze** on the last frame, and the traffic between the two ends should stop. On the server, the transfer should break rather than complete.

5 Actual Outcome and Observations

The attack behaved exactly as expected in both topologies.

5.1 LAN scenario

The client streamed the video from the Windows server (Figure 3). The attacker poisoned both hosts and injected resets (Figures 4–5); a live capture shows the poisoned ARP and the forged RST on the wire (Figure 6). The moment the resets landed, the video **froze** with a buffering spinner (Figure 7).

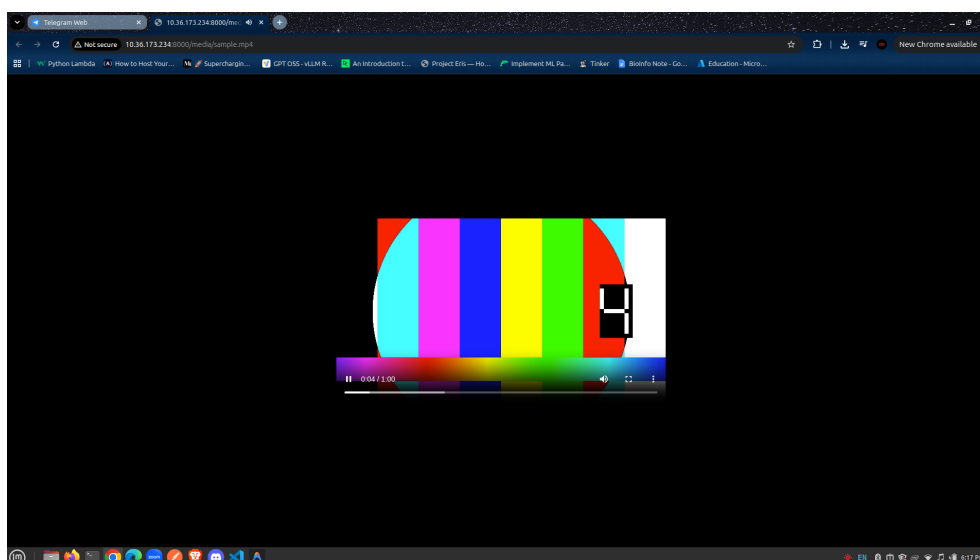


Figure 3: LAN: the video streaming normally from the server (10.36.173.234:8000) before the attack.

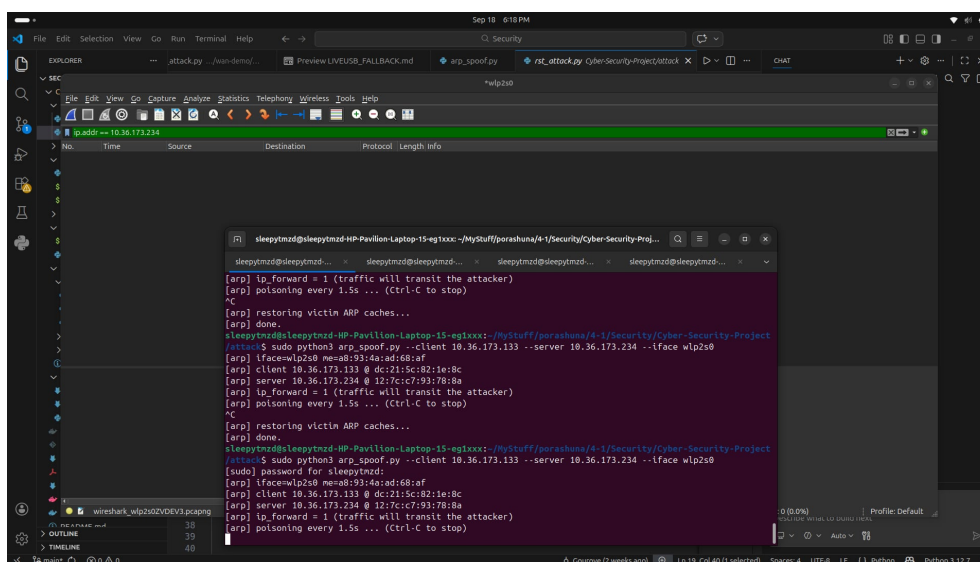


Figure 4: LAN attacker: ARP spoofing the server and the client.

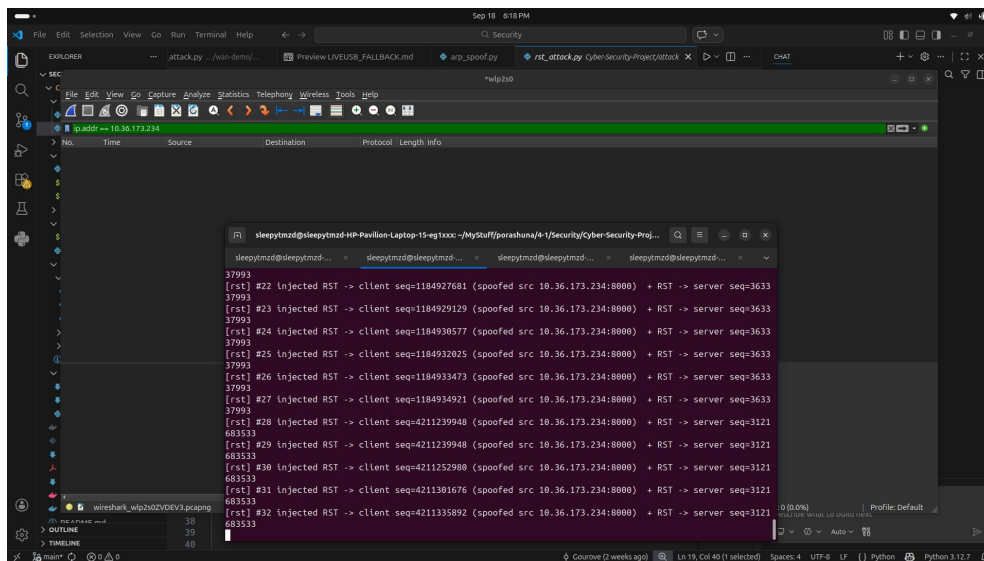


Figure 5: LAN attacker: injecting the forged TCP resets into the video connection.

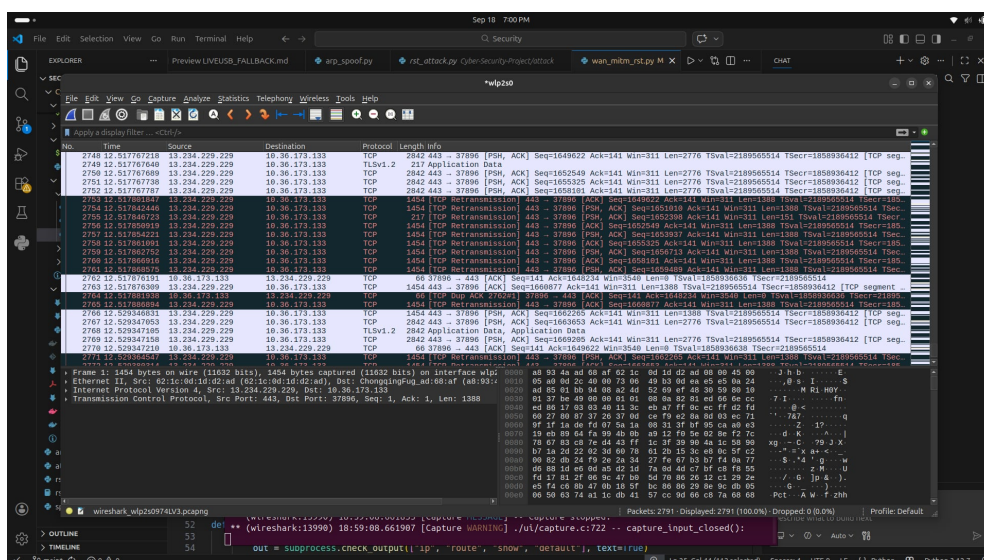


Figure 6: LAN: Wireshark capture showing the ARP poisoning and the injected RST.

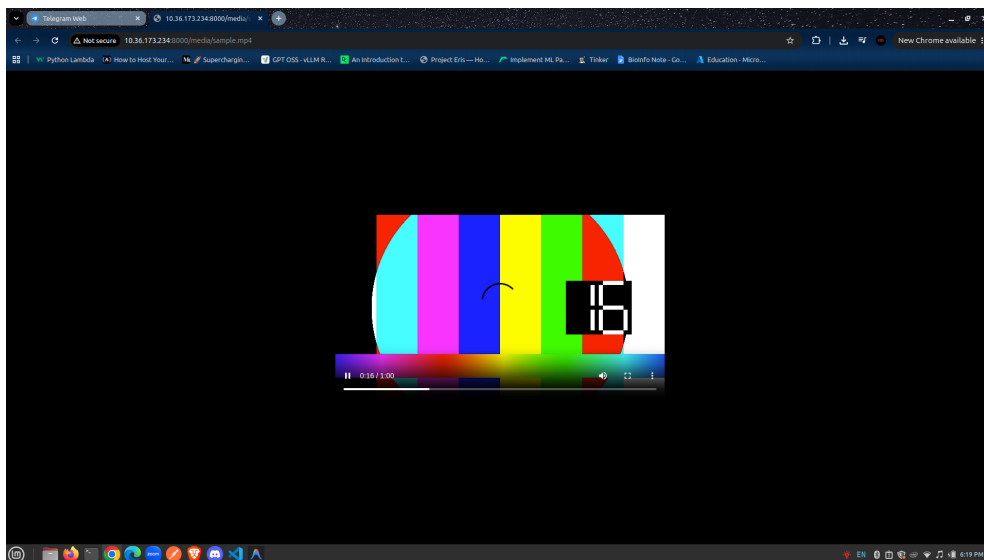


Figure 7: LAN result: the video frozen (buffering spinner) after the reset injection.

5.2 WAN scenario

The client played the video from the remote ngrok server over HTTPS (Figure 8). We first read the current ngrok edge IP with Wireshark (Figure 9), then poisoned the client and gateway and injected resets (Figures 10–11); the capture shows the forged reset on the encrypted (443) flow (Figure 12). The stream froze on command. This confirms that the reset works beneath TLS.

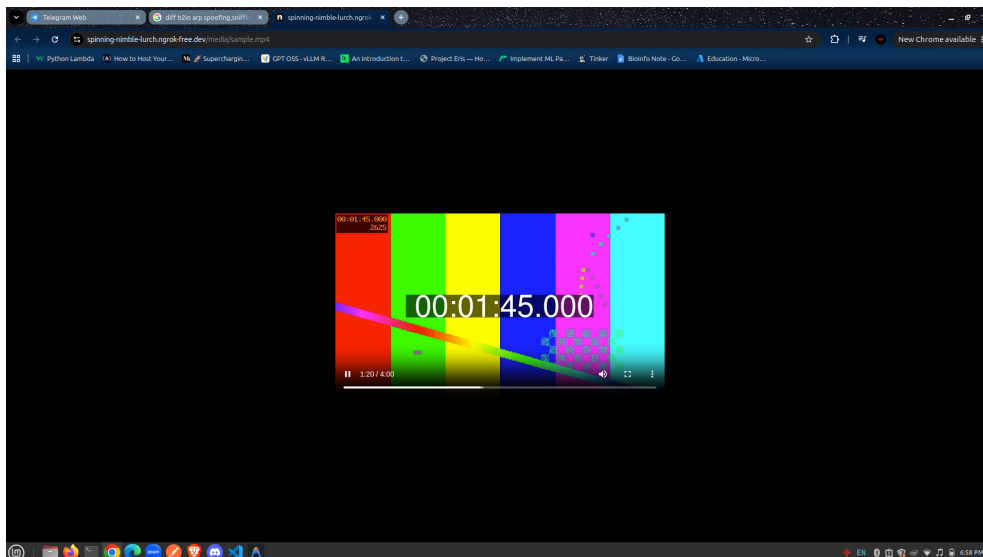


Figure 8: WAN: the video streaming from the remote ngrok server (spinning-nimble-lurch.ngrok-free.dev) over HTTPS.

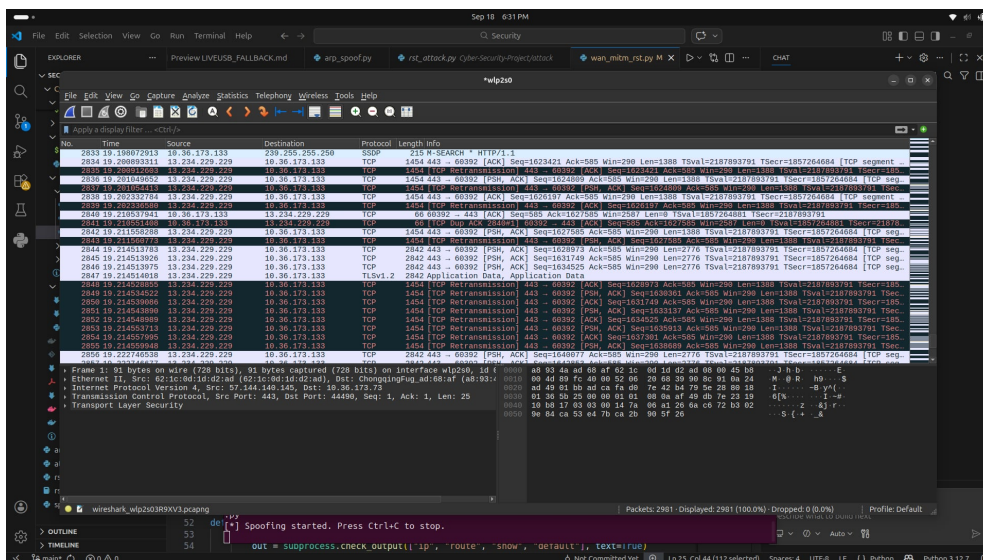


Figure 9: WAN: using Wireshark to read the current ngrok edge (server) IP for the video flow.

Key result

Both the LAN and WAN attacks succeeded on real hardware. In the WAN case the stream was HTTPS, yet the reset still froze it. This shows that a transport-layer reset is not stopped by application-layer encryption.

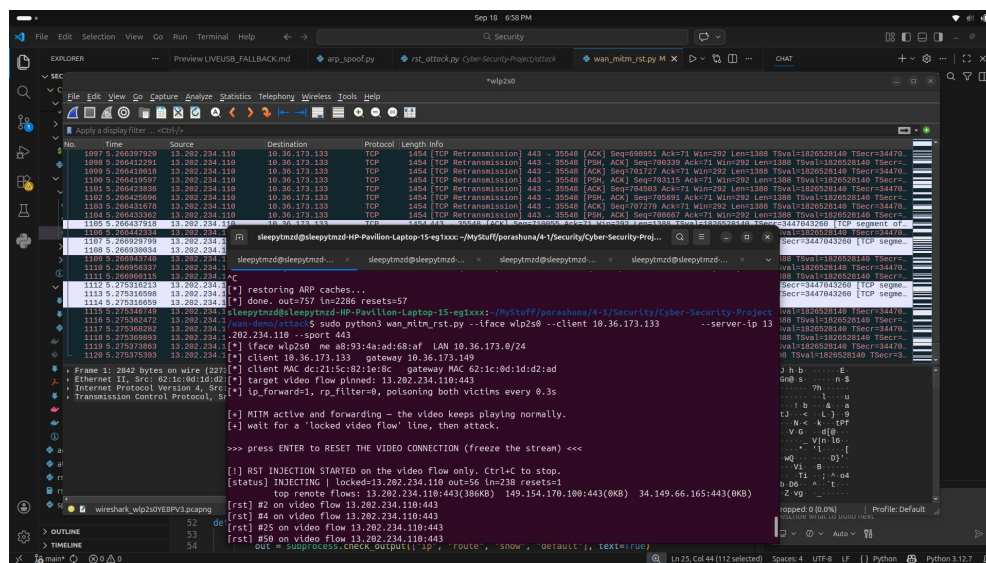


Figure 10: WAN attacker: ARP spoofing the client and the gateway (the phone hotspot).

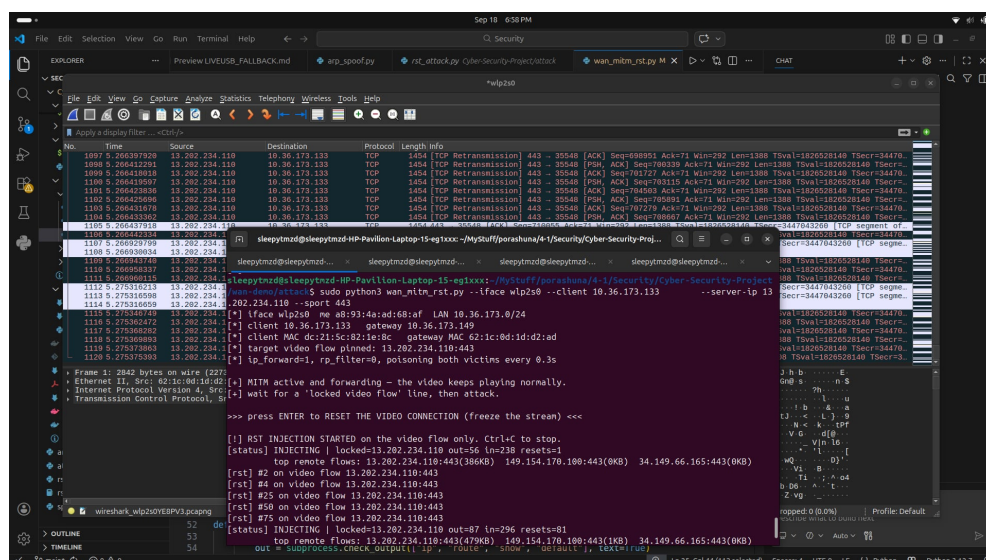


Figure 11: WAN attacker: injecting the forged resets into the client's HTTPS video connection.

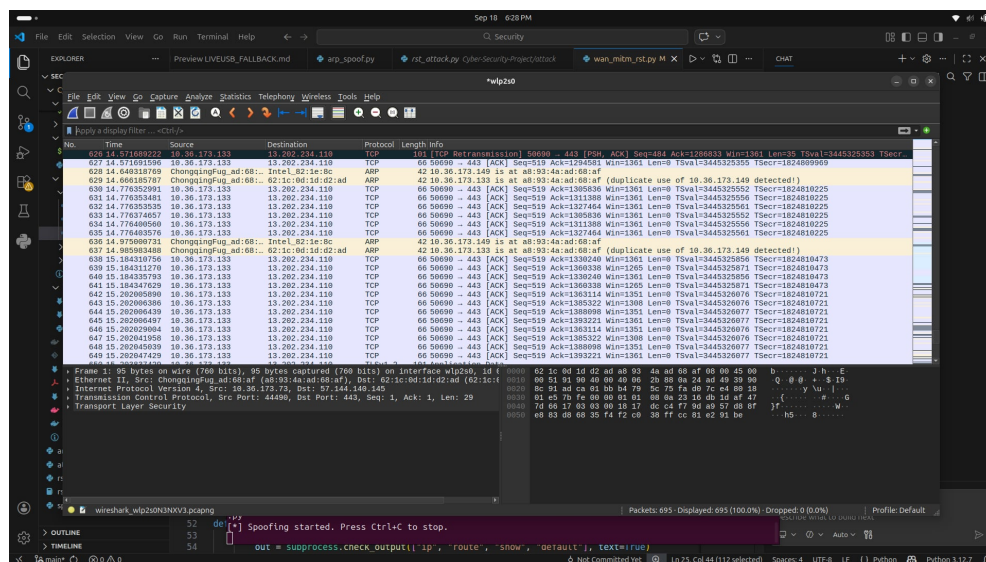


Figure 12: WAN: Wireshark capture of the injected RST on the port-443 (HTTPS) flow.

6 Expected vs. Actual: Differences and Why

The final outcome matched the expectation in both topologies. The practical observations and deviations along the way were:

- **HTTPS does not stop the attack.** Confirmed on the WAN run: the RST is in the TCP header, below TLS.
- **ngrok load-balances and rotates IPs.** The public host resolves to several edge IPs and they change on each restart, so we identify the live one with Wireshark per session and target that (Figure 9).
- **Exact, no-guess resets.** Reading ack (for the sender) and seq+len (for the receiver) from observed packets makes every reset land on the first try, satisfying RFC 5961's exact-sequence rule.
- **Address family matters.** If the client used IPv6, our IPv4 ARP/RST attack would not see the flow; we ensured the client used IPv4.
- **Detector tuning (an important correction).** An early detector version false-positived on ordinary traffic: it treated IP-ID 0 as a "crafted packet" (but modern Linux/Windows use IP-ID 0 normally) and compared a RST's TTL against the *wrong* direction (client TTL 64 vs. server TTL 127). We fixed it so the **ARP spoof is the primary signal**, and a RST is judged only *while a MITM is active* and only against its *own* direction's TTL. It is now silent on normal traffic and precise under attack (Figure 14).

7 Defence Mechanism

The defence has two parts: **detect** the attack, and **prevent** it.

7.1 Detection

An IDS on the client raises two alarms. The *primary, reliable* one is the ARP anomaly: a stable IP's MAC changing, or one MAC suddenly answering for several IPs (Figures 13 and 15). The corroborating one is the injected reset itself. This is flagged only while a MITM is active, and only when its TTL does not match the same direction's data packets (the genuine data is forwarded and loses a TTL hop, while the forged reset is minted locally), as in Figure 14.

The screenshot shows a VS Code editor with a Python script named `detector.py` in the center. The script defines a `Detector` class with a `def on_arp(self, pkt):` method. The terminal window on the right displays the output of the script, showing several alerts related to ARP spoofing. The alerts indicate that a MAC address is claiming both victim IPs and that the server's MAC address has changed.

```
class Detector:
    def on_arp(self, pkt):
        return
        prev = self.ip2mac.get(ip)
        if prev and prev != mac:
            self.mitm_notify()
```

```
[ALERT #1] ARP SPOOF / MITM: one MAC a8:93:4a:ad:68:af now claims IPs ['10.36.173.73', '10.36.173.234']
[ALERT #2] ARP SPOOF / MITM: one MAC a8:93:4a:ad:68:af now claims IPs ['10.36.173.73', '10.36.173.234']
[ALERT #3] ARP SPOOF / MITM: 10.36.173.234 changed MAC a8:93:4a:ad:68:af -> 12:7c:c7:93:78:8a
[ALERT #4] ARP SPOOF / MITM: 10.36.173.234 changed MAC 12:7c:c7:93:78:8a -> a8:93:4a:ad:68:af
[ALERT #5] ARP SPOOF / MITM: one MAC a8:93:4a:ad:68:af now claims IPs ['10.36.173.73', '10.36.173.234']
```

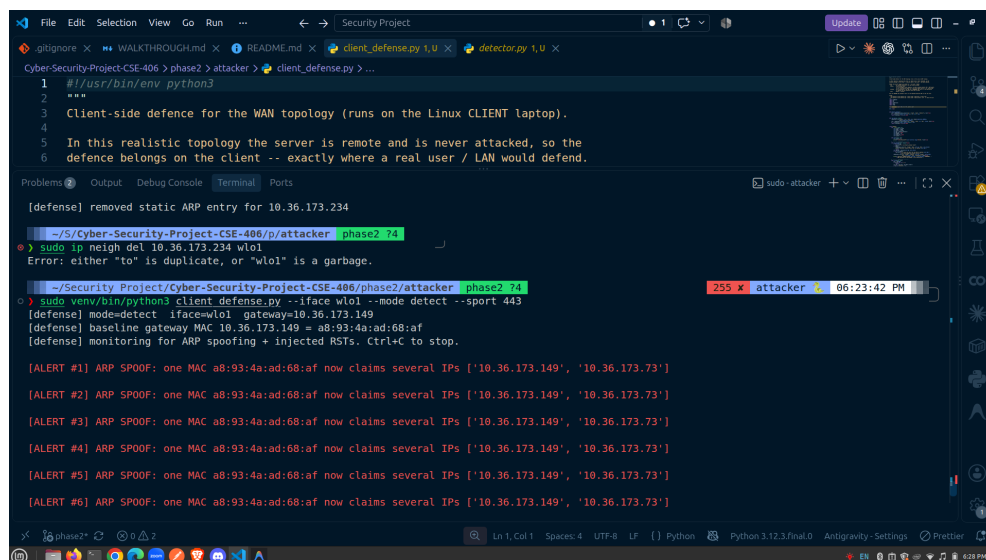
Figure 13: LAN detection: the IDS flags the ARP spoofing (one MAC claiming both victim IPs; the server's MAC changing).

The screenshot shows a VS Code editor with a Python script named `detector.py` in the center. The script defines a `Detector` class with a `def on_arp(self, pkt):` method. The terminal window on the right displays the output of the script, showing several alerts related to injected TCP RST packets. The alerts indicate that the injected RST is flagged by per-direction TTL (TTL 64 != data TTL 127, minted locally).

```
class Detector:
    def on_arp(self, pkt):
        return
        prev = self.ip2mac.get(ip)
        if prev and prev != mac:
            self.mitm_notify()
```

```
[ALERT #63] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48296 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.98s into an active stream
[ALERT #64] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48304 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.04s into an active stream
[ALERT #65] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48304 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.05s into an active stream
[ALERT #66] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48304 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.10s into an active stream
[ALERT #67] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48304 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.16s into an active stream
[ALERT #68] INJECTED TCP RST 10.36.173.234:8000 -> 10.36.173.133:48304 | TTL 64 != this-direction data TTL 127 (RST minted locally, not forwarded); 0.16s into an active stream
```

Figure 14: LAN detection: the injected RST flagged by per-direction TTL (TTL 64 != data TTL 127, minted locally).



```
File Edit Selection View Go Run ... Security Project
CyberSecurity-Project-CSE-406 > phase2 > attacker > client_defense.py > ...
1 #!/usr/bin/env python3
2
3 Client-side defence for the WAN topology (runs on the Linux CLIENT laptop).
4
5 In this realistic topology the server is remote and is never attacked, so the
6 defence belongs on the client -- exactly where a real user / LAN would defend.

Problems Output Debug Console Terminal Ports
[defense] removed static ARP entry for 10.36.173.234

~/Security-Project-CSE-406/p/attacker phase2 74
> sudo ip neigh del 10.36.173.234 wlol
Error: either "to" is duplicate, or "wlol" is a garbage.

~/Security Project/Cyber-Security-Project-CSE-406/phase2/attacker phase2 74
> sudo venv/bin/python3 client_defense.py --iface wlol --mode detect --sport 443
[defense] mode=detect iface=wlol gateway=10.36.173.149
[defense] baseline gateway MAC 10.36.173.149 = a8:93:4a:ad:68:af
[defense] monitoring for ARP spoofing + injected RSTs. Ctrl+C to stop.

[ALERT #1] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']
[ALERT #2] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']
[ALERT #3] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']
[ALERT #4] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']
[ALERT #5] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']
[ALERT #6] ARP SPOOF: one MAC a8:93:4a:ad:68:af now claims several IPs ['10.36.173.149', '10.36.173.73']

phase2* 0 2
Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Python 3.12.3.final.0 AntigraVity-Settings Prettier
6:28 PM
```

Figure 15: WAN detection: the client-side IDS flags the ARP spoofing of the client and gateway.

7.2 Prevention

LAN, static ARP (fully prevents). Because we own both endpoints, pinning each other's real MAC as a permanent ARP entry (on the Linux client with `ip neigh`, on the Windows server with `netsh`) makes the poisoned replies ineffective. The attacker never gets on-path, so no valid reset can be built and the video keeps playing under attack (Figure 16).

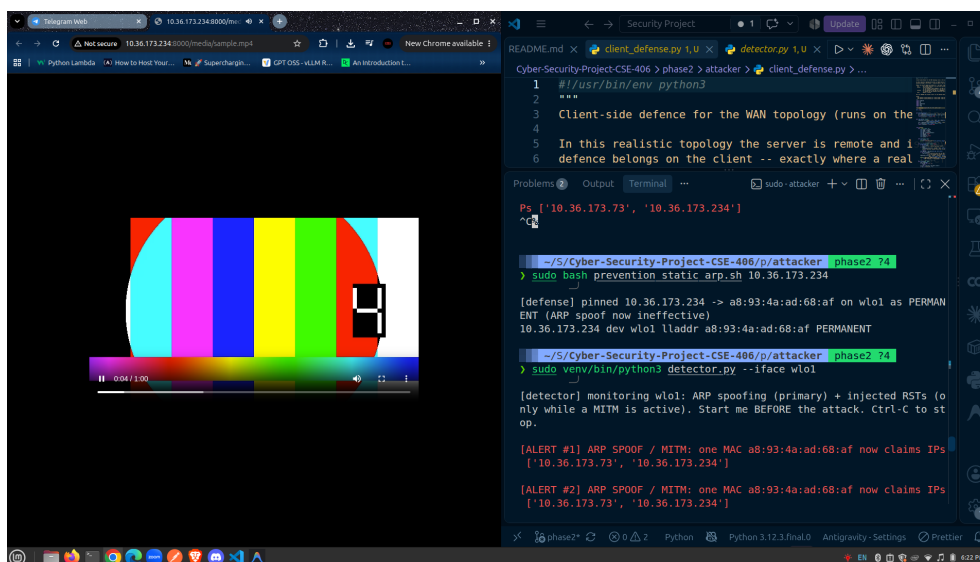


Figure 16: LAN prevention: with the peer's MAC pinned PERMANENT ("ARP spoof now ineffective"), the video keeps playing even while the attacker is poisoning.

WAN, VPN (fully prevents). On the hotspot the gateway is someone's phone, which we do not control, so static ARP on the client alone is not sufficient (the attacker can still poison the gateway and see the downstream). The correct fix is a client-side **VPN** (WireGuard/UDP): all traffic is tunnelled, so there is no exposed TCP connection to reset. With the VPN enabled the attack no longer freezes the stream.

Why encryption alone is not enough

TLS/HTTPS hides the *payload* but leaves the outer TCP header in the clear, so an on-path attacker can still reset it (we demonstrated this on the WAN run). Prevention requires removing the MITM (static ARP, when you own both ends) or protecting the transport itself (VPN/IPsec).

7.3 Does the defence work as expected?

Yes. Detection fired correctly in both topologies and stayed silent on normal traffic after the fix. Prevention fully stopped the attack: static ARP on the LAN (video kept playing), and a VPN on the WAN. One nuance is itself a useful finding: client-only static ARP cannot fully prevent the attack when you do not control the gateway.

8 Assumptions

1. **Same L2 segment:** attacker, client and (LAN) server share one broadcast domain.
2. **No client isolation:** the network allows station-to-station traffic (true for the phone hotspot; false for most campus/enterprise Wi-Fi).

3. **Raw packet access** on the attacker (Linux + Scapy, root).
4. **Victims accept the forged ARP** and update their caches.
5. **On-path exact sequence:** being MITM, the attacker knows the exact sequence/ack, so RFC 5961 accepts the reset without a challenge ACK.
6. **Video over TCP (IPv4):** a TCP reset applies; a UDP transport (e.g. a VPN or QUIC) would not be affected.

9 Conclusion

We implemented the TCP reset attack on a video stream with our own code and demonstrated it end-to-end on two real topologies: a local LAN and a realistic WAN setup with a remote server. In both, the man-in-the-middle was established by ARP spoofing and the video froze on command, including over HTTPS in the WAN case, which confirms that encryption does not stop a transport-layer reset. We also built and demonstrated the defence: an IDS that reliably detects the attack (after correcting its heuristics to avoid false positives), static-ARP prevention that fully blocks it on the LAN, and a VPN-based prevention for the WAN. In short, preventing this attack requires either removing the attacker's foothold on a network you control, or protecting the transport end-to-end, not merely encrypting the payload.

All experiments were carried out on networks the team owns, for the coursework demonstration only.